ARM Interrupt & Exception Architecture (Unified Workflow Document)

GICv2 / GICv3 • ARMv8-A • QNX Neutrino • BSP/Firmware

1. CPU Event Classification

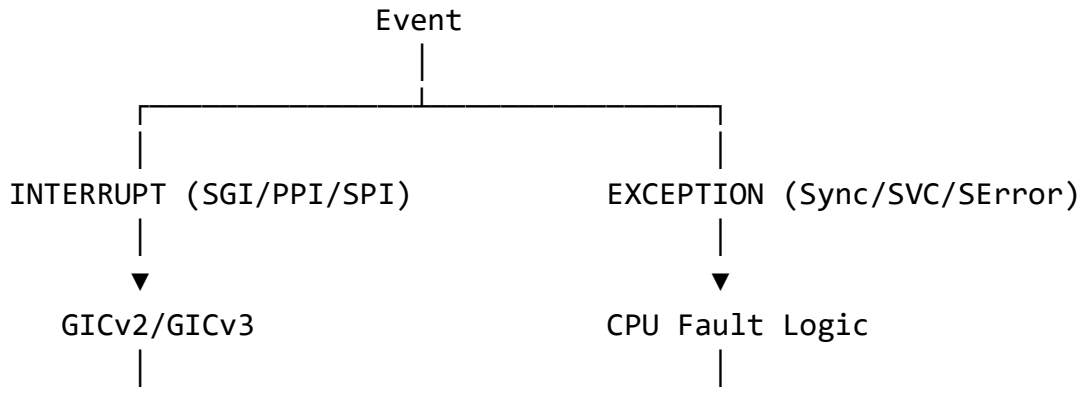
Interrupts (Asynchronous, from GIC)

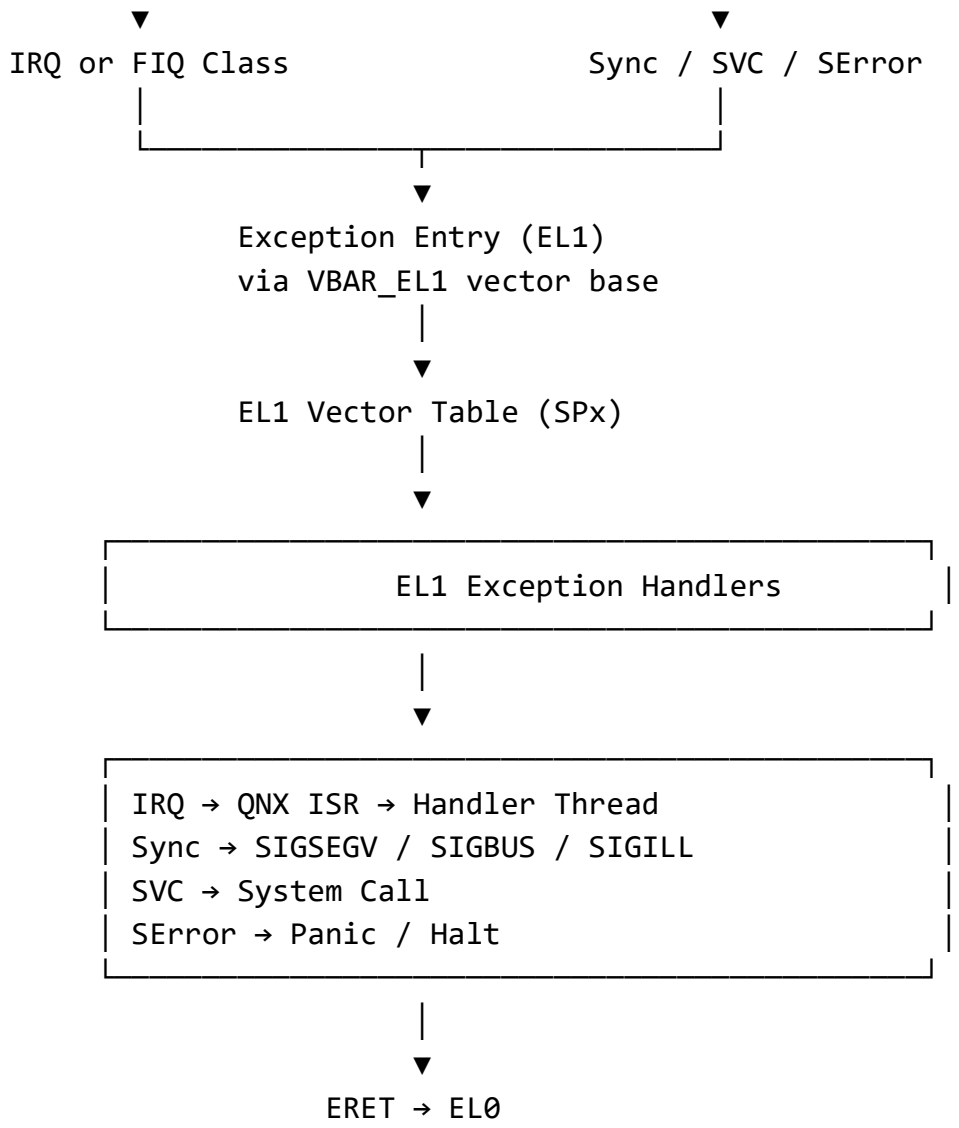
- **SGI** – Software Generated Interrupt (IPI)
- **PPI** – Per-Processor Interrupt
- **SPI** – Shared Peripheral Interrupt
- **IRQ** – Normal interrupt exception class
- **FIQ** – Fast interrupt exception class (GICv2 only)

Exceptions (Synchronous + Asynchronous CPU faults)

- **Sync** – Faults (abort, alignment, illegal instruction)
- **SVC** – System call
- **IRQ** – Interrupt exception class
- **FIQ** – Fast interrupt exception class
- **SError** – Fatal asynchronous hardware error

2. Unified Interrupt & Exception Workflow





3. Interrupt Source Types (GIC-Level)

```
SGI → Software interrupt (IPI)
PPI → Per-core interrupt (timer, watchdog)
SPI → External device interrupt (GPIO, PCIe, USB)
```

These define **where** the interrupt originates.

4. IRQ vs FIQ (CPU-Level Exception Classes)

```
IRQ → Normal interrupt exception
FIQ → Fast interrupt exception (higher priority, GICv2 only)
```

These define **how** the CPU classifies the interrupt.

5. How Interrupts Become Exceptions

```
SGI/PPI/SPI
        ↓
GIC Distributor + CPU Interface
        ↓
IRQ or FIQ signal to CPU
        ↓
CPU enters exception mode
        ↓
Exception vector lookup (VBAR_EL1)
        ↓
EL1 handler executes
```

6. Exception Vector Table (EL1)

QNX uses **SPx entries only**.

```
VBAR_EL1
│
├── SP0 Branch (Unused)
│      ├── Sync (0x000)
│      ├── IRQ  (0x080)
│      ├── FIQ  (0x100)
│      └── SError (0x180)
│
└── SPx Branch (Used)
       ├── Sync (0x200)
       ├── IRQ  (0x280)
```

```
        ├── FIQ  (0x300)
        └── SError (0x380)
```

7. QNX Interrupt Handling Workflow

```
SGI/PPI/SPI
        ↓
IRQ exception entry (0x280)
        ↓
QNX ISR (interrupt context)
        - Read interrupt ID
        - Clear device status
        - Post pulse/event
        - Write EOI
        ↓
Handler Thread (thread context)
        - Full processing
        - IPC, memory, logic
```

8. QNX Exception Handling Workflow

```
EL0 executes instruction
        ↓
Exception occurs (Sync/SVC/IRQ/SError)
        ↓
CPU saves:
    SPSR_EL1 ← PSTATE
    ELR_EL1  ← PC
        ↓
Vector lookup via VBAR_EL1
        ↓
EL1 handler:
    Sync → SIGSEGV/SIGBUS/SIGILL
    SVC  → system call
    IRQ  → ISR → handler thread
    SError → panic
        ↓
```

```
ERET → return to EL0
```

9. Unified Architecture Summary (One View)

```
SGI/PPI/SPI → GIC → IRQ/FIQ → EL1 Vector → ISR/Handler
Sync/SVC/SError → CPU → EL1 Vector → Fault Handler
```

Everything ultimately flows through:

- **VBAR_EL1**
- **Exception classes**
- **EL1 vector table**
- **QNX kernel handlers**